

SIMATS ENGINEERING ASSESSMENT TOOL 2

Name: E.V. Mahendra Reddy (192425214)

COURSE CODE: MLA03

**COURSE NAME: Reinforcement learning for
Environment and Agent**

COURSE OUTCOME COVERED

CO3: Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)

Assessment Tool Weightage: 35%

Dr G. A. SENTHIL

QUESTIONS: 10

Total

Marks: 50

Annexure A

Experiments

Duration: 3hrs

1. Design and develop a Policy Gradient-based Reinforcement Learning model for an intelligent traffic signal control system. Implement the solution using Python and TensorFlow/PyTorch, compare **REINFORCE** and **A2C**, and evaluate average vehicle waiting time, throughput, and convergence rate.

**(10
Marks)**

Answer:

An intelligent traffic signal controller can be modeled as a policy-gradient RL agent that observes traffic conditions at a junction and chooses signal phases to minimize vehicle waiting time and maximize throughput.

Problem Formulation (RL Environment Design)

- **State space:** queue length and average waiting time per lane, current signal phase, and time elapsed in the current phase (a numeric vector, e.g. from SUMO traffic simulator or a custom grid simulation).
- **Action space:** discrete — choose which signal phase to activate next (e.g., North-South green, East-West green, left-turn phase).
- **Reward function:** negative of cumulative vehicle waiting time (or queue length) after each step, so the agent is rewarded for keeping queues short; a bonus term can be added for throughput (vehicles cleared).

Algorithm Comparison

- **REINFORCE:** a Monte-Carlo policy-gradient method that updates the policy using the full-episode return; simple to implement but has high variance and needs full episodes before each update, so convergence is slow.

- **A2C (Advantage Actor-Critic):** adds a critic (value network) to estimate a baseline, updating the policy using the advantage $A(s,a)=R-V(s)$ after every few steps (n-step or per-step); this reduces variance and converges faster and more stably than REINFORCE.

Sample Implementation (Python / PyTorch)

```
import torch, torch.nn as nn, torch.optim as optim
import numpy as np

class PolicyNet(nn.Module):
    def __init__(self, state_dim, n_actions):
        super().__init__()
        self.fc = nn.Sequential(nn.Linear(state_dim, 64),
                                nn.ReLU(),
                                nn.Linear(64, 64), nn.ReLU())
        self.pi = nn.Linear(64, n_actions)      # actor head
    def forward(self, x):
        return torch.softmax(self.pi(self.fc(x)), dim=-1)

class ValueNet(nn.Module):
    def __init__(self, state_dim):
        super().__init__()
        self.fc = nn.Sequential(nn.Linear(state_dim, 64),
                                nn.ReLU(),
                                nn.Linear(64, 1))
    def forward(self, x):
        return self.fc(x)

# --- REINFORCE update (Monte-Carlo) ---
def reinforce_update(policy, opt, log_probs, rewards,
                    gamma=0.99):
    G, returns = 0, []
    for r in reversed(rewards):
```

```

    G = r + gamma * G
    returns.insert(0, G)
    returns = torch.tensor(returns)
    returns = (returns - returns.mean()) / (returns.std() + 1e-8)
    loss = -torch.stack([lp * G for lp, G in zip(log_probs,
returns)]).sum()
    opt.zero_grad(); loss.backward(); opt.step()

# --- A2C update (per step, with critic baseline) ---
def a2c_update(policy, value, opt_p, opt_v, log_prob, reward,
s, s_next, done, gamma=0.99):
    v_s = value(s)
    v_next = value(s_next).detach() * (1 - done)
    target = reward + gamma * v_next
    advantage = (target - v_s).detach()
    actor_loss = -log_prob * advantage
    critic_loss = (target - v_s).pow(2)
    opt_p.zero_grad();
actor_loss.backward(retain_graph=True); opt_p.step()
    opt_v.zero_grad(); critic_loss.backward(); opt_v.step()

# TrafficEnv() would expose step(action)->(next_state,
reward, done)
# built on SUMO / a custom queueing simulation of the
junction.

```

Evaluation & Expected Outcome

Both agents are trained over many simulated episodes and compared on average vehicle waiting time, throughput (vehicles/hour), and convergence rate (episodes to reach stable reward). A2C is expected to converge faster and achieve lower variance in waiting time than REINFORCE because the critic's baseline reduces gradient noise, though REINFORCE

may still reach a comparable final policy given enough training episodes.

2. Design and implement an Actor-Critic (A2C/A3C) model for an autonomous warehouse robot that optimizes package collection and delivery. Compare the performance of **Vanilla Policy Gradient** and **Actor-Critic** algorithms based on reward, training stability, and task completion time.

**(10
Marks)**

Answer:

An autonomous warehouse robot that collects and delivers packages can be trained with policy-gradient RL to learn efficient pick-up and drop-off routes while avoiding collisions.

Problem Formulation (RL Environment Design)

- **State space:** robot's (x,y) position/orientation, positions of pending pickup and delivery locations, obstacle/shelf map, and battery/time remaining.
- **Action space:** discrete movement commands (up, down, left, right, pick, drop) or continuous velocity commands for a more realistic robot base.
- **Reward function:** positive reward for successful pickup/delivery, small negative step penalty to encourage efficiency, and a larger negative penalty for collisions or exceeding a time limit.

Algorithm Comparison

- **Vanilla Policy Gradient:** directly optimizes $E[\sum \log \pi(a|s) \cdot G]$ using full-episode Monte-Carlo returns with no learned baseline (or a simple moving-average baseline);

easy to implement but suffers from high variance and slow, unstable training, especially as warehouse layouts grow larger.

- **Actor-Critic (A2C/A3C):** uses a critic to bootstrap value estimates, enabling per-step (or n-step) updates instead of waiting for full episodes; A3C additionally runs multiple parallel agent workers asynchronously, further stabilizing training and speeding up wall-clock convergence in large warehouse state spaces.

Sample Implementation (Python / PyTorch)

```
import torch, torch.nn as nn, torch.optim as optim

class ActorCritic(nn.Module):
    def __init__(self, state_dim, n_actions):
        super().__init__()
        self.shared = nn.Sequential(nn.Linear(state_dim, 128),
nn.ReLU())
        self.actor = nn.Linear(128, n_actions)
        self.critic = nn.Linear(128, 1)
    def forward(self, x):
        h = self.shared(x)
        return torch.softmax(self.actor(h), dim=-1), self.critic(h)

def train_episode(env, model, opt, gamma=0.99,
use_baseline=True):
    state = env.reset()
    log_probs, values, rewards = [], [], []
    done = False
    while not done:
        s = torch.tensor(state, dtype=torch.float32)
        probs, value = model(s)
        dist = torch.distributions.Categorical(probs)
```

```

    action = dist.sample()
    next_state, reward, done, _ = env.step(action.item())
    log_probs.append(dist.log_prob(action))
    values.append(value)
    rewards.append(reward)
    state = next_state

G, returns = 0, []
for r in reversed(rewards):
    G = r + gamma * G
    returns.insert(0, G)
returns = torch.tensor(returns, dtype=torch.float32)
values = torch.cat(values).squeeze()

if use_baseline:                                # Actor-Critic (A2C)
    advantage = returns - values.detach()
    actor_loss = -(torch.stack(log_probs) * advantage).sum()
    critic_loss = (returns - values).pow(2).sum()
    loss = actor_loss + 0.5 * critic_loss
else:                                            # Vanilla Policy Gradient
    loss = -(torch.stack(log_probs) * returns).sum()

opt.zero_grad(); loss.backward(); opt.step()
return sum(rewards)

```

Evaluation & Expected Outcome

Training curves are compared on cumulative reward per episode, variance of reward across runs, and average task-completion time (steps to finish all deliveries). The Actor-Critic variant typically shows a smoother, faster-rising reward curve and lower completion time variance than the Vanilla Policy Gradient baseline, confirming the benefit of the learned value-function baseline.

3. Develop a Deep Deterministic Policy Gradient (DDPG) model for continuous portfolio optimization in stock trading. Design the state space, action space, reward function, and evaluate cumulative return, risk, and convergence performance.

**(10
Marks)**

Answer:

Continuous portfolio optimization — deciding what fraction of capital to allocate to each asset at every time step — is naturally suited to DDPG, an off-policy actor-critic algorithm designed for continuous action spaces.

Problem Formulation (RL Environment Design)

- **State space:** a window of recent price/return history, technical indicators (moving averages, volatility, RSI) for each asset, and the current portfolio weights.
- **Action space:** a continuous vector of portfolio weights (one per asset, softmax-normalized to sum to 1) representing capital allocation for the next period.
- **Reward function:** the portfolio's realized return over the period, optionally penalized by a risk term such as volatility or transaction costs, e.g. $\text{reward} = \text{return} - \lambda \cdot \text{risk_penalty} - \text{transaction_cost}$.

Algorithm Comparison

- **DDPG core idea:** combines a deterministic actor $\mu(s|\theta)$ producing continuous actions with a critic $Q(s,a|\phi)$ estimating action value; it uses target networks (slowly-updated copies) and an experience replay buffer for stable off-policy learning, plus exploration noise (e.g., Ornstein-Uhlenbeck) added to the deterministic action.

- **Why not discrete policy gradients here:** discretizing continuous portfolio weights loses precision and scales poorly with the number of assets, whereas DDPG directly outputs continuous weight vectors, making it well suited to this problem.

Sample Implementation (Python / PyTorch)

```
import torch, torch.nn as nn, torch.optim as optim, random
from collections import deque

class Actor(nn.Module):
    def __init__(self, state_dim, action_dim):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(state_dim, 128),
                                nn.ReLU(),
                                nn.Linear(128, 128), nn.ReLU(),
                                nn.Linear(128, action_dim))
    def forward(self, s):
        return torch.softmax(self.net(s), dim=-1)  # weights
sum to 1

class Critic(nn.Module):
    def __init__(self, state_dim, action_dim):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(state_dim+action_dim,
128), nn.ReLU(),
                                nn.Linear(128, 128), nn.ReLU(),
                                nn.Linear(128, 1))
    def forward(self, s, a):
        return self.net(torch.cat([s, a], dim=-1))

replay = deque(maxlen=100000)

def ddpg_update(actor, critic, actor_t, critic_t, opt_a, opt_c,
```

```

        batch_size=64, gamma=0.99, tau=0.005):
    batch = random.sample(replay, batch_size)
    s, a, r, s2, d = map(lambda x: torch.tensor(x,
dtype=torch.float32), zip(*batch))

    with torch.no_grad():
        a2 = actor_t(s2)
        target_q = r.unsqueeze(1) + gamma * (1 -
d.unsqueeze(1)) * critic_t(s2, a2)
        critic_loss = ((critic(s, a) - target_q) ** 2).mean()
        opt_c.zero_grad(); critic_loss.backward(); opt_c.step()

    actor_loss = -critic(s, actor(s)).mean()
    opt_a.zero_grad(); actor_loss.backward(); opt_a.step()

    for tp, p in zip(actor_t.parameters(), actor.parameters()):
        tp.data.copy_(tau * p.data + (1 - tau) * tp.data)
    for tp, p in zip(critic_t.parameters(), critic.parameters()):
        tp.data.copy_(tau * p.data + (1 - tau) * tp.data)

```

Evaluation & Expected Outcome

The DDPG agent is evaluated using cumulative return over a backtesting period, the Sharpe ratio / maximum drawdown as risk measures, and convergence of the training reward curve across episodes. A well-tuned DDPG agent should show steadily increasing cumulative return with acceptable risk (bounded drawdown) and a critic loss that decreases and stabilizes, indicating convergence of the value estimate.

4. Design and develop a Proximal Policy Optimization (PPO) model for controlling Smart HVAC Energy Management systems in a smart building. Implement the

model to minimize energy consumption while maintaining occupant comfort, and compare PPO with REINFORCE.

(10

Marks)

Answer:

Smart HVAC energy management can be framed as an RL control problem where the agent adjusts heating/cooling setpoints to minimize energy consumption while keeping occupants comfortable.

Problem Formulation (RL Environment Design)

- **State space:** current indoor temperature and humidity, outdoor weather, occupancy count/schedule, time of day, and current HVAC setpoint.
- **Action space:** discrete or continuous adjustment of the temperature setpoint (e.g., -2°C to $+2^{\circ}\text{C}$) or HVAC power level.
- **Reward function:** $\text{reward} = -(\text{energy_consumed}) - \lambda \cdot \text{comfort_violation_penalty}$, where the comfort penalty grows if indoor temperature drifts outside a comfortable band (e.g., $21\text{--}24^{\circ}\text{C}$).

Algorithm Comparison

- **REINFORCE:** updates the policy using full-episode Monte-Carlo returns; because HVAC episodes (e.g., a full day) are long, the high variance of Monte-Carlo returns makes learning slow and noisy.
- **PPO (Proximal Policy Optimization):** uses a clipped surrogate objective to limit how far each policy update can move from the previous policy, combined with a critic for advantage estimation (GAE); this yields much more stable, sample-efficient training and is far less sensitive to

learning-rate choice than REINFORCE, making it better suited to the long-horizon HVAC task.

Sample Implementation (Python / PyTorch)

```
import torch, torch.nn as nn, torch.optim as optim

class ActorCritic(nn.Module):
    def __init__(self, state_dim, n_actions):
        super().__init__()
        self.body = nn.Sequential(nn.Linear(state_dim, 64),
nn.Tanh())
        self.actor = nn.Linear(64, n_actions)
        self.critic= nn.Linear(64, 1)
    def forward(self, x):
        h = self.body(x)
        return torch.softmax(self.actor(h), dim=-1), self.critic(h)

def ppo_update(model, opt, states, actions, old_log_probs,
returns, advantages,
        clip_eps=0.2, epochs=4):
    for _ in range(epochs):
        probs, values = model(states)
        dist = torch.distributions.Categorical(probs)
        new_log_probs = dist.log_prob(actions)
        ratio = torch.exp(new_log_probs - old_log_probs)

        surr1 = ratio * advantages
        surr2 = torch.clamp(ratio, 1 - clip_eps, 1 + clip_eps) *
advantages
        actor_loss = -torch.min(surr1, surr2).mean()
        critic_loss = (returns - values.squeeze()).pow(2).mean()
        entropy_bonus = dist.entropy().mean()

        loss = actor_loss + 0.5 * critic_loss - 0.01 *
```

```
entropy_bonus
    opt.zero_grad(); loss.backward(); opt.step()

# REINFORCE (for comparison) uses the same actor but a
# single full-episode
# Monte-Carlo update with no clipping and no multiple epochs
# per batch.
```

Evaluation & Expected Outcome

Both agents are compared on total daily energy consumption (kWh), percentage of time spent within the comfort band, and reward-curve convergence speed. PPO is expected to reduce energy consumption while maintaining comfort more reliably, with a smoother and faster-converging training curve than REINFORCE due to its clipped, multi-epoch updates on the same batch of experience.

- 5. Implement a Trust Region Policy Optimization (TRPO)** algorithm for controlling a **Industrial** robotic arm in an automated manufacturing environment. Evaluate policy stability, precision, convergence speed, and overall production efficiency.

**(10
Marks)**

Answer:

Controlling an industrial robotic arm for precise, repeatable manufacturing tasks (e.g., pick-and-place or welding) demands a policy-update method that avoids large, destabilizing policy changes — making TRPO a natural fit.

Problem Formulation (RL Environment Design)

- **State space:** joint angles and angular velocities of the arm, end-effector position/orientation, and target position (goal) in the workspace.
- **Action space:** continuous torque or joint-velocity commands for each of the arm's degrees of freedom.
- **Reward function:** negative distance between the end-effector and the target position, with penalties for excessive joint torque/jerk (for smooth, safe motion) and a bonus for successful task completion within tolerance.

Algorithm Comparison

- **TRPO core idea:** maximizes the policy's expected advantage subject to a KL-divergence constraint between the old and new policy (trust region), guaranteeing monotonic policy improvement and preventing the large, destabilizing updates that plain policy gradient methods can make — critical when a bad update could send a physical arm into an unsafe configuration.
- **Trade-off:** TRPO's constrained optimization (using conjugate gradient and a line search) is more computationally expensive per update than PPO's simple clipped objective, but it offers stronger theoretical guarantees on policy stability, which is valuable in precision manufacturing.

Sample Implementation (Python / PyTorch)

```
import torch, torch.nn as nn
from torch.distributions import Normal

class GaussianPolicy(nn.Module):
    def __init__(self, state_dim, action_dim):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(state_dim, 64),
```

```

nn.Tanh(),
            nn.Linear(64, 64), nn.Tanh())
    self.mu    = nn.Linear(64, action_dim)
    self.log_std = nn.Parameter(torch.zeros(action_dim))
    def forward(self, s):
        h = self.net(s)
        return self.mu(h), self.log_std.exp()

def surrogate_loss(policy, states, actions, advantages,
old_log_probs):
    mu, std = policy(states)
    dist = Normal(mu, std)
    log_probs = dist.log_prob(actions).sum(-1)
    ratio = torch.exp(log_probs - old_log_probs)
    return (ratio * advantages).mean()

def kl_divergence(policy, old_mu, old_std, states):
    mu, std = policy(states)
    kl = torch.log(std / old_std) + (old_std.pow(2) + (old_mu -
mu).pow(2)) / (2 * std.pow(2)) - 0.5
    return kl.sum(-1).mean()

# TRPO update (conceptual outline):
# 1. Compute policy gradient g of the surrogate loss.
# 2. Use conjugate gradient to solve  $Fx = g$  ( $F$  = Fisher
information matrix).
# 3. Compute step direction and perform a backtracking line
search that
#   enforces  $KL(\text{old\_policy} \parallel \text{new\_policy}) \leq \delta$  (trust
region size).
# In practice, libraries such as `garage` or `stable-baselines3`
# (TRPO via sb3-contrib) implement this efficiently.

```

Evaluation & Expected Outcome

Performance is evaluated on end-effector positioning precision (mm error), policy stability (variance of KL-divergence per update, ensuring it stays within the trust region), convergence speed (episodes to reach target precision), and overall production efficiency (successful picks per hour). TRPO is expected to yield smoother, monotonically improving performance with fewer catastrophic drops in policy quality compared to unconstrained policy-gradient methods.

6. Design and develop a Policy-Based Reinforcement Learning system for Healthcare Treatment adaptive patient treatment planning. Compare **A2C** and **PPO** in terms of treatment effectiveness, cumulative reward, and learning stability using simulated patient data.

**(10
Marks)**

Answer:

Adaptive patient treatment planning can be modeled as a sequential decision process where an RL agent recommends treatment actions (e.g., drug dosage or therapy choice) at each stage based on the patient's evolving simulated health state.

Problem Formulation (RL Environment Design)

- **State space:** patient vitals and biomarkers (e.g., blood pressure, glucose level, lab results), treatment history, and disease progression stage.
- **Action space:** discrete treatment options (e.g., dosage levels, therapy types) or continuous dosage amounts.
- **Reward function:** positive reward for improvement in health indicators toward a target range, negative reward for adverse events or side effects, and a terminal reward for full recovery or penalty for treatment failure.

Algorithm Comparison

- **A2C:** synchronous actor-critic that updates the policy using single-step or n-step advantage estimates; simpler and faster per-update but can be less sample-efficient and more sensitive to step-size, which matters when patient simulation data is limited.
- **PPO:** improves on A2C by clipping the policy update ratio, allowing multiple epochs of stable minibatch updates on the same collected trajectories; this typically yields more robust and safer policy improvements — an important property in healthcare, where large erratic updates to a treatment policy are undesirable.

Sample Implementation (Python / PyTorch)

```
import torch, torch.nn as nn

class PolicyValueNet(nn.Module):
    def __init__(self, state_dim, n_actions):
        super().__init__()
        self.shared = nn.Sequential(nn.Linear(state_dim, 64),
nn.ReLU())
        self.actor = nn.Linear(64, n_actions)
        self.critic = nn.Linear(64, 1)
    def forward(self, x):
        h = self.shared(x)
        return torch.softmax(self.actor(h), dim=-1), self.critic(h)

def compute_gae(rewards, values, gamma=0.99, lam=0.95):
    advantages, gae = [], 0
    values = values + [0]
    for t in reversed(range(len(rewards))):
        delta = rewards[t] + gamma * values[t+1] - values[t]
        gae = delta + gamma * lam * gae
        advantages.insert(0, gae)
```

```

return advantages

# A2C: single pass, immediate update using 1-step / n-step
advantage
def a2c_step(model, opt, log_prob, advantage, value, target):
    actor_loss = -log_prob * advantage.detach()
    critic_loss = (target - value).pow(2)
    loss = actor_loss + 0.5 * critic_loss
    opt.zero_grad(); loss.backward(); opt.step()

# PPO: multiple clipped-epoch updates over a batch of
trajectories
def ppo_step(model, opt, states, actions, old_log_probs,
advantages, returns, eps=0.2):
    for _ in range(4):
        probs, values = model(states)
        dist = torch.distributions.Categorical(probs)
        ratio = torch.exp(dist.log_prob(actions) - old_log_probs)
        clipped = torch.clamp(ratio, 1-eps, 1+eps) * advantages
        loss = -torch.min(ratio*advantages, clipped).mean() + \
            0.5*(returns - values.squeeze()).pow(2).mean()
        opt.zero_grad(); loss.backward(); opt.step()

```

Evaluation & Expected Outcome

Using simulated patient trajectories, treatment effectiveness (fraction of patients reaching target health outcomes), cumulative reward, and learning stability (variance of reward across training runs/seeds) are compared. PPO typically achieves higher and more stable cumulative reward with lower run-to-run variance than A2C, supporting its use where safe, conservative policy updates matter, though A2C may train faster in wall-clock time for simpler patient models.

7. Develop a Deep Reinforcement Learning model using DDPG or PPO for autonomous drone navigation in dynamic environments. Evaluate navigation accuracy, obstacle avoidance, energy consumption, and policy convergence.

**(10
Marks)**

Answer:

Autonomous drone navigation in dynamic environments requires continuous control of thrust/orientation while reacting to moving obstacles, making it suitable for either DDPG (deterministic continuous control) or PPO (stable stochastic policy optimization); PPO is chosen below for its robustness to environment dynamics.

Problem Formulation (RL Environment Design)

- **State space:** drone position, velocity, orientation (from IMU/GPS), relative position/velocity of nearby obstacles (from LIDAR/camera), and distance/direction to the goal.
- **Action space:** continuous control signals — thrust and roll/pitch/yaw rate commands (or simplified 2D/3D velocity commands).
- **Reward function:** positive reward for progress toward the goal and successful arrival, negative reward proportional to proximity to obstacles (collision risk), a penalty for excessive energy use, and a large negative terminal reward for collision.

Algorithm Comparison

- **DDPG:** deterministic actor-critic well suited to continuous actions, sample-efficient via replay buffer reuse, but can be unstable/sensitive to hyperparameters and exploration noise in highly dynamic, fast-changing obstacle fields.

- **PPO:** stochastic policy with clipped updates naturally handles the continuous action space (via a Gaussian policy head) and tends to be more robust to noisy, non-stationary dynamics such as moving obstacles, at the cost of requiring more environment interactions than off-policy DDPG.

Sample Implementation (Python / PyTorch)

```
import torch, torch.nn as nn
from torch.distributions import Normal

class DronePolicy(nn.Module):
    def __init__(self, state_dim, action_dim):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(state_dim, 128),
                                nn.Tanh(),
                                nn.Linear(128, 128), nn.Tanh())
        self.mu = nn.Linear(128, action_dim)
        self.log_std = nn.Parameter(torch.zeros(action_dim) -
0.5)
        self.value = nn.Linear(128, 1)
    def forward(self, s):
        h = self.net(s)
        return self.mu(h), self.log_std.exp(), self.value(h)

def collect_action(policy, state):
    mu, std, value = policy(state)
    dist = Normal(mu, std)
    action = dist.sample()
    return action, dist.log_prob(action).sum(-1), value

def ppo_clip_update(policy, opt, states, actions,
old_log_probs, advantages, returns, eps=0.2):
    mu, std, values = policy(states)
    dist = Normal(mu, std)
```

```

new_log_probs = dist.log_prob(actions).sum(-1)
ratio = torch.exp(new_log_probs - old_log_probs)
surr = torch.min(ratio*advantages, torch.clamp(ratio, 1-eps,
1+eps)*advantages)
loss = -surr.mean() + 0.5*(returns -
values.squeeze()).pow(2).mean()
opt.zero_grad(); loss.backward(); opt.step()
# DroneEnv.step(action) simulates dynamics (e.g. via AirSim /
PyBullet /
# a custom kinematic model) and returns next_state, reward,
done.

```

Evaluation & Expected Outcome

The trained agent is evaluated on navigation accuracy (final distance to goal), obstacle-avoidance success rate (percentage of collision-free episodes), energy consumption (integrated thrust over the trajectory), and policy convergence (reward curve over training episodes). PPO is generally expected to give a higher collision-free success rate and steadier convergence in dynamic obstacle fields than a naively-tuned DDPG agent, though DDPG can match or exceed it in static or slowly-changing environments given careful hyperparameter tuning.

- 8. Design and implement an A3C-based Reinforcement Learning** framework for dynamic cloud resource allocation. Compare the algorithm with Vanilla Policy Gradient using metrics such as response time, CPU utilization, and resource efficiency.

(10 Marks)

Answer:

Dynamic cloud resource allocation — deciding how to assign CPU/memory/VMs to incoming workloads — is a high-throughput, continuously-arriving-request problem well suited to A3C's parallel, asynchronous training.

Problem Formulation (RL Environment Design)

- **State space:** current CPU/memory/network utilization per server/VM, incoming request queue length and resource demand, and historical load pattern (recent time window).
- **Action space:** discrete choice of which server/VM (or resource tier) to allocate an incoming request to, or how much resource quota to assign.
- **Reward function:** $\text{reward} = -(\text{response_time}) - \lambda \cdot (\text{resource_wastage}) + \mu \cdot (\text{utilization_efficiency})$, balancing fast response with efficient use of available compute resources and penalizing overload/underuse.

Algorithm Comparison

- **Vanilla Policy Gradient:** a single agent learns from full-episode Monte-Carlo returns; in a fast-changing cloud environment, sample collection is slow and the high variance of a single trajectory's returns makes convergence unreliable.
- **A3C (Asynchronous Advantage Actor-Critic):** runs many parallel actor-learner workers, each interacting with its own copy of the environment and asynchronously updating a shared global network; this decorrelates training data (removing the need for a replay buffer), massively speeds up data collection, and stabilizes learning — well matched to the high-throughput, parallel nature of cloud workloads.

Sample Implementation (Python / PyTorch)

```

import torch, torch.nn as nn, torch.multiprocessing as mp

class GlobalNet(nn.Module):
    def __init__(self, state_dim, n_actions):
        super().__init__()
        self.shared = nn.Sequential(nn.Linear(state_dim, 128),
nn.ReLU())
        self.actor = nn.Linear(128, n_actions)
        self.critic = nn.Linear(128, 1)
    def forward(self, x):
        h = self.shared(x)
        return torch.softmax(self.actor(h), dim=-1), self.critic(h)

def worker(global_net, opt, worker_id, env_fn, gamma=0.99,
n_steps=20):
    local_net = GlobalNet(*global_net.dims)
    local_net.load_state_dict(global_net.state_dict())
    env = env_fn()
    state = env.reset()
    while True:
        log_probs, values, rewards = [], [], []
        for _ in range(n_steps):
            s = torch.tensor(state, dtype=torch.float32)
            probs, value = local_net(s)
            dist = torch.distributions.Categorical(probs)
            action = dist.sample()
            next_state, reward, done, _ = env.step(action.item())
            log_probs.append(dist.log_prob(action));
values.append(value); rewards.append(reward)
            state = env.reset() if done else next_state
            if done: break

        R = 0 if done else local_net(torch.tensor(state,

```

```

dtype=torch.float32))[1].item()
    returns = []
    for r in reversed(rewards):
        R = r + gamma * R
        returns.insert(0, R)
    returns = torch.tensor(returns)
    values = torch.cat(values).squeeze()
    advantage = returns - values
    loss =
-(torch.stack(log_probs)*advantage.detach()).sum() +
0.5*advantage.pow(2).sum()

    opt.zero_grad(); loss.backward()
    for lp, gp in zip(local_net.parameters(),
global_net.parameters()):
        gp._grad = lp.grad
    opt.step()
    local_net.load_state_dict(global_net.state_dict()) # sync

# Launch several worker processes sharing `global_net` and a
shared optimizer.

```

Evaluation & Expected Outcome

The two approaches are compared on average request response time, CPU/resource utilization efficiency, and training wall-clock time to reach a stable policy. A3C is expected to converge substantially faster in wall-clock time (thanks to parallel data collection) and to achieve lower response times and higher resource-utilization efficiency than the single-worker Vanilla Policy Gradient baseline.

9. Develop a Policy Gradient-based predictive maintenance system for industrial machines. Design the RL environment,

reward function, and policy network to minimize maintenance costs and machine downtime. Compare the performance of **REINFORCE** and **PPO**.

(10
Marks)

Answer:

A predictive maintenance system can be modeled as an RL agent that periodically decides whether to continue operating a machine, schedule maintenance, or replace a component, based on the machine's degrading condition, to minimize downtime and maintenance cost.

Problem Formulation (RL Environment Design)

- **State space:** sensor readings (vibration, temperature, pressure), estimated remaining useful life (RUL), time since last maintenance, and machine operating mode.
- **Action space:** discrete actions — {continue operating, schedule preventive maintenance, perform maintenance now, replace component}.
- **Reward function:** small positive reward per step of continued healthy operation, a moderate negative reward for the cost of performing maintenance, and a large negative reward for unplanned downtime/failure.

Algorithm Comparison

- **REINFORCE:** learns directly from full-episode (e.g., full equipment lifecycle) Monte-Carlo returns; because failures are rare events, the sparse, high-variance reward signal makes REINFORCE slow to learn an effective maintenance-timing policy.
- **PPO:** uses advantage estimates and clipped multi-epoch updates on batches of transitions, making far more efficient use of the (often limited and expensive-to-generate)

machine failure data, and produces a more stable policy that avoids drastically changing its maintenance schedule between updates.

Sample Implementation (Python / PyTorch)

```
import torch, torch.nn as nn

class MaintenancePolicy(nn.Module):
    def __init__(self, state_dim, n_actions):
        super().__init__()
        self.body = nn.Sequential(nn.Linear(state_dim, 64),
nn.ReLU(), nn.Linear(64, 64), nn.ReLU())
        self.actor = nn.Linear(64, n_actions)
        self.critic = nn.Linear(64, 1)
    def forward(self, x):
        h = self.body(x)
        return torch.softmax(self.actor(h), dim=-1), self.critic(h)

def reinforce_loss(log_probs, rewards, gamma=0.99):
    G, returns = 0, []
    for r in reversed(rewards):
        G = r + gamma*G; returns.insert(0, G)
    returns = torch.tensor(returns)
    returns = (returns - returns.mean()) / (returns.std() + 1e-8)
    return -(torch.stack(log_probs) * returns).sum()

def ppo_loss(model, states, actions, old_log_probs,
advantages, returns, eps=0.2):
    probs, values = model(states)
    dist = torch.distributions.Categorical(probs)
    ratio = torch.exp(dist.log_prob(actions) - old_log_probs)
    clipped = torch.clamp(ratio, 1-eps, 1+eps) * advantages
    return -torch.min(ratio*advantages, clipped).mean() +
0.5*(returns-values.squeeze()).pow(2).mean()
```

MaintenanceEnv simulates component degradation (e.g. a stochastic
RUL decay model) and returns reward/downtime signals per
decision step.

Evaluation & Expected Outcome

Both agents are evaluated over many simulated equipment lifecycles on total maintenance cost, cumulative unplanned downtime, and reward-curve convergence speed. PPO is expected to converge to a policy with lower total cost and downtime, and with more consistent (lower-variance) performance across runs, compared to REINFORCE, which tends to need many more episodes to reach a comparably good maintenance-timing policy.

10. Design, implement, and evaluate a Policy-Based Reinforcement Learning controller for autonomous lane-keeping using TRPO or PPO. Compare the selected algorithm with A2C based on lane deviation, safety, reward convergence, and training efficiency.

**(10
Marks)**

Answer:

Autonomous lane-keeping requires a controller that produces smooth, safe steering corrections; PPO is used here (as a practical, easier-to-tune alternative to TRPO with similar stability benefits) and compared against A2C.

Problem Formulation (RL Environment Design)

- **State space:** lateral offset from the lane center, heading angle relative to the lane, vehicle speed, and curvature of

the road ahead (from camera/lane-detection or a simulator such as CARLA).

- **Action space:** continuous steering angle command (and optionally throttle/brake for speed regulation).
- **Reward function:** $\text{reward} = -|\text{lateral_offset}| - \lambda \cdot |\text{heading_error}| - \text{penalty_for_large_steering_jerk}$, with a large negative terminal reward for leaving the lane/road (safety violation).

Algorithm Comparison

- **PPO / TRPO:** both use a trust-region-like constraint (explicit KL constraint for TRPO, clipped ratio for PPO) to prevent large, unsafe policy updates — important because an overly aggressive update could produce erratic steering behaviour; PPO is simpler to implement and tune while retaining most of TRPO's stability benefit.
- **A2C:** updates the policy using simple advantage estimates without any trust-region safeguard, so it can occasionally make larger, less controlled policy updates, leading to more variable lane-keeping behaviour (higher variance in lane deviation) during training compared to PPO/TRPO.

Sample Implementation (Python / PyTorch)

```
import torch, torch.nn as nn
from torch.distributions import Normal

class SteeringPolicy(nn.Module):
    def __init__(self, state_dim):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(state_dim, 64),
nn.Tanh(), nn.Linear(64, 64), nn.Tanh())
        self.mu = nn.Linear(64, 1) # steering angle
        self.log_std = nn.Parameter(torch.zeros(1) - 1.0)
```

```

        self.value = nn.Linear(64, 1)
    def forward(self, s):
        h = self.net(s)
        return self.mu(h), self.log_std.exp(), self.value(h)

def ppo_lane_update(policy, opt, states, actions,
old_log_probs, advantages, returns, eps=0.2):
    mu, std, values = policy(states)
    dist = Normal(mu, std)
    new_log_probs = dist.log_prob(actions).sum(-1)
    ratio = torch.exp(new_log_probs - old_log_probs)
    clipped = torch.clamp(ratio, 1-eps, 1+eps) * advantages
    loss = -torch.min(ratio*advantages, clipped).mean() +
0.5*(returns - values.squeeze()).pow(2).mean()
    opt.zero_grad(); loss.backward(); opt.step()

def a2c_lane_update(policy, opt, log_prob, value, reward,
next_value, done, gamma=0.99):
    target = reward + gamma * next_value * (1 - done)
    advantage = (target - value).detach()
    loss = -log_prob * advantage + 0.5 * (target - value).pow(2)
    opt.zero_grad(); loss.backward(); opt.step()
# LaneKeepEnv (e.g. built on CARLA / a bicycle-model
simulator) returns
# lateral offset, heading error, and speed as state, and
computes reward.

```

Evaluation & Expected Outcome

The controllers are compared on average absolute lane deviation, safety (number of lane-departure/off-road events per 1000 steps), reward convergence speed, and training efficiency (episodes needed to reach a stable, safe policy). PPO/TRPO is expected to achieve lower average lane deviation and fewer safety violations with a smoother

convergence curve than A2C, reflecting the value of trust-region-constrained updates for safety-critical continuous control.

Code of Conduct:

I _E.V.Mahendra Reddy_(192425214) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts

Experimental Design	30	Well-planned , systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

